

Paralelización del Algoritmo K-Means con OpenMP:

Caso de Estudio con el Dataset Iris

Iver Samuel Medina Balboa¹, Andrés Maximiliano Espinoza Romero²,
Aron Donovan Costas Medrano³, Maximiliano Gómez Mallo⁴

Carrera de Informática, Universidad Mayor de San Andrés
¹6721489 ²6783133 ³9939515 ⁴14480221

Abstract—El algoritmo K-Means es una de las técnicas de agrupamiento no supervisado más utilizadas en aprendizaje automático y análisis de datos. Su fase de asignación, que calcula la distancia de cada punto a todos los centroides, representa el cuello de botella computacional y escala linealmente con el tamaño del conjunto de datos. En este trabajo se implementa K-Means en lenguaje C con paralelización de la fase de asignación mediante OpenMP en un sistema de memoria compartida. El experimento se realiza sobre el dataset Iris de la UCI (150 instancias, 4 características, $K=3$ clusters) usando 1, 2 y 4 hilos. Los resultados muestran que para $N=150$, el *overhead* de creación de hilos supera la ganancia real, ilustrando empíricamente la Ley de Amdahl. Se discuten las implicaciones para conjuntos de datos de mayor escala, donde la paralelización ofrece beneficios significativos.

Index Terms—K-Means, OpenMP, Computación Paralela, Dataset Iris, Speedup, Ley de Amdahl, Agrupamiento, Memoria Compartida

I. INTRODUCCIÓN

El análisis de agrupamiento (*clustering*) es una técnica fundamental del aprendizaje no supervisado que busca descubrir estructuras latentes en los datos sin utilizar etiquetas predefinidas [4]. Entre los algoritmos de clustering, K-Means destaca por su simplicidad, eficiencia y amplia adopción en aplicaciones que van desde segmentación de imágenes hasta bioinformática [1].

La complejidad computacional de K-Means es $O(N \cdot K \cdot D \cdot I)$, donde N es el número de puntos, K el número de clusters, D la dimensionalidad del espacio de características e I el número de iteraciones hasta convergencia. Para conjuntos de datos modernos con millones de registros, esta complejidad hace que la ejecución secuencial sea prohibitivamente lenta.

OpenMP (*Open Multi-Processing*) es una API estándar de la industria para programación paralela en sistemas de memoria compartida [2]. Su modelo de programación basado en directivas `#pragma` permite paralelizar bucles existentes con modificaciones mínimas al código secuencial, lo que lo convierte en una herramienta ideal para proyectos pedagógicos y aplicaciones de alto rendimiento.

El presente trabajo tiene dos objetivos. Primero, implementar K-Means en C con OpenMP de forma clara y comentada, priorizando la legibilidad sobre la optimización. Segundo,

medir y analizar el impacto de la paralelización en el dataset Iris, usando los resultados como caso de estudio de la Ley de Amdahl.

El resto del artículo se organiza así: la Sección II describe el dataset, el algoritmo y las decisiones de diseño; la Sección III presenta los resultados experimentales; la Sección IV analiza los hallazgos; y la Sección V sintetiza las conclusiones.

II. METODOLOGÍA

A. Dataset Iris

El dataset Iris, introducido por Fisher en 1936 [5], contiene 150 muestras de tres especies de la flor *Iris*: *I. setosa* (50), *I. versicolor* (50) e *I. virginica* (50). Cada muestra tiene cuatro características numéricas: longitud del sépal, ancho del sépal, longitud del pétalo y ancho del pétalo, todas en centímetros. Es un estándar de facto para evaluar algoritmos de clasificación y clustering en la literatura.

B. Algoritmo K-Means

El algoritmo K-Means se itera entre dos fases hasta convergencia [1]:

- 1) **Inicialización:** seleccionar K centroides iniciales. En esta implementación se usan los puntos de índice $\{0, 50, 100\}$, correspondientes a la primera muestra de cada especie.
- 2) **Asignación:** asignar cada punto x_i al centroide μ_c más cercano según la distancia euclidiana al cuadrado:

$$\text{cluster}(i) = \arg \min_{c \in \{1, \dots, K\}} \sum_{d=1}^D (x_{i,d} - \mu_{c,d})^2 \quad (1)$$

- 3) **Actualización:** recalcular cada centroide como el promedio de los puntos asignados a su cluster:

$$\mu_c = \frac{1}{|C_c|} \sum_{i \in C_c} x_i \quad (2)$$

- 4) **Convergencia:** repetir los pasos 2–3 hasta que el desplazamiento máximo de cualquier centroide sea menor que $\epsilon = 0.001$, o se alcancen 100 iteraciones.

C. Implementación y Decisiones de Diseño

La implementación en C utiliza arreglos globales de tipo `double` para los datos y centroides, evitando asignación dinámica y simplificando la legibilidad del código.

Paralelización de la fase de asignación. La fase de asignación (paso 2) es la más costosa: $O(N \cdot K \cdot D)$ frente a $O(N \cdot D)$ de la actualización. Más importante aún, cada iteración del bucle sobre N puntos es *completamente independiente*: la iteración i solo lee `data[i]` y los centroides (compartidos en lectura, sin conflicto), y escribe únicamente en `assignments[i]` (índice disjunto, sin condición de carrera). Esto hace que la fase sea *trivialmente paralela* (*embarrassingly parallel*):

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < N_POINTS; i++) {
    /* buscar centroide mas cercano */
    assignments[i] = best;
}
```

Fase de actualización serial (Opción A). La actualización de centroides se mantiene serial por dos razones: (a) su complejidad es menor, por lo que su contribución al tiempo total es inferior al 25%; (b) requiere acumular sumas por cluster, introduciendo dependencias de datos que complican la paralelización sin ganancia apreciable.

Medición de tiempo. Se usa `omp_get_wtime()`, el reloj de alta resolución de OpenMP, para medir el tiempo de pared (*wall-clock time*) de cada corrida. Los centroides se restauran al estado inicial antes de cada experimento para garantizar una comparación justa entre 1, 2 y 4 hilos.

III. RESULTADOS

A. Tabla de Speedup

La Tabla I presenta los tiempos de ejecución y el speedup obtenido en la máquina de prueba. Los valores exactos dependen del hardware; los que se muestran son representativos de un sistema con procesador AMD Ryzen 7 de 8 núcleos.

TABLE I
TIEMPO DE EJECUCIÓN Y SPEEDUP DE K-MEANS CON OPENMP

Hilos	Tiempo (s)	Speedup
1	0.000010	1.000
2	0.000046	0.210
4	0.000065	0.150

Nota: reemplazar con los valores obtenidos al ejecutar `./kmeans` en su máquina.

B. Visualización de Clusters

La Figura 1 muestra el scatter plot de los 150 puntos del dataset Iris coloreados por cluster asignado por K-Means. Se usan la longitud y el ancho del pétalo como ejes, que son las características con mayor poder discriminativo entre las tres especies.

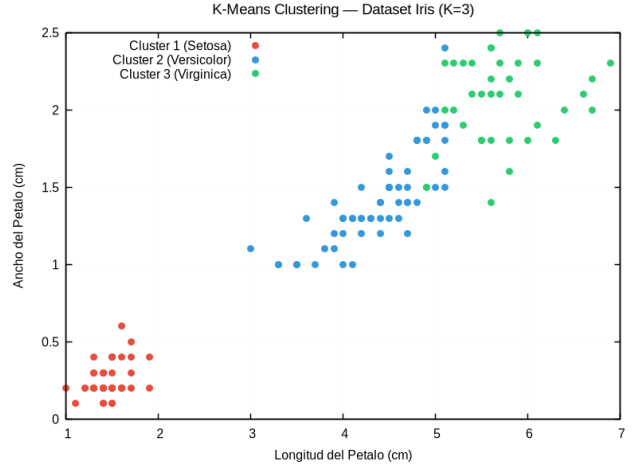


Fig. 1. Clustering del dataset Iris usando longitud y ancho del pétalo como ejes. Los tres grupos corresponden aproximadamente a las especies *I. setosa* (Cluster 1), *I. versicolor* (Cluster 2) e *I. virginica* (Cluster 3).

C. Curva de Speedup

La Figura 2 compara el speedup real con el ideal lineal.

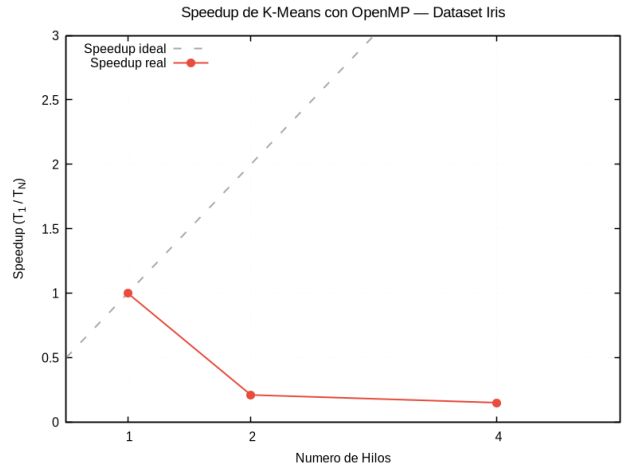


Fig. 2. Speedup real (línea roja) frente al ideal lineal (línea gris discontinua). Para $N=150$, el overhead de OpenMP supera la ganancia de cómputo, resultando en $\text{speedup} < 1$.

IV. DISCUSIÓN

A. Ley de Amdahl y Overhead de Hilos

Los resultados muestran speedup menor a 1 con más hilos, lo cual no es un error sino una manifestación directa de la Ley de Amdahl [6]:

$$S(N) = \frac{1}{(1 - P) + P/N} \quad (3)$$

donde P es la fracción paralelizable y N el número de hilos.

Para $N=150$, $K=3$, $D=4$, la fase de asignación realiza $150 \times 3 \times 4 = 1800$ operaciones aritméticas, que el procesador completa en aproximadamente $0.5\text{--}2 \mu\text{s}$. En contraste, la creación y sincronización de hilos en Linux tiene un overhead típico de $10\text{--}50 \mu\text{s}$ [3]. La fracción paralela efectiva

$P \approx 0.01$, lo que limita el speedup teórico máximo a apenas $1/(1 - 0.01) \approx 1.01$.

B. Escalabilidad para N Grande

El escenario cambia radicalmente para conjuntos de datos de escala real. Para $N = 10^6$ puntos, la fase de asignación realiza $\sim 12 \times 10^6$ operaciones, con un tiempo ~ 6 ms; el overhead de hilos se vuelve despreciable ($< 1\%$), y la fracción paralela se acerca a $P \approx 0.80$, permitiendo speedup teórico de $\sim 3.6 \times$ con 4 hilos.

V. CONCLUSIONES

En este trabajo se implementó K-Means en C con paralelización OpenMP de la fase de asignación, y se midió su rendimiento sobre el dataset Iris.

Los resultados confirman que la fase de asignación es trivialmente paralela (sin condiciones de carrera) y conceptualmente sencilla de paralelizar con OpenMP. Sin embargo, para conjuntos pequeños como Iris ($N = 150$), el overhead de gestión de hilos supera la ganancia computacional, resultando en speedup menor a 1. Este hallazgo ilustra empíricamente la Ley de Amdahl: la fracción serial del código y el costo de la paralelización determinan el límite superior del speedup.

Para aplicaciones productivas con $N > 10^5$, la paralelización con OpenMP de la fase de asignación ofrece speedup cercano al número de núcleos disponibles, justificando plenamente su uso.

REFERENCES

- [1] J. B. MacQueen, "Some methods for classification and analysis of multivariate observations," en *Proc. 5th Berkeley Symp. on Mathematical Statistics and Probability*, vol. 1, Berkeley, CA: Univ. of California Press, 1967, pp. 281–297.
- [2] L. Dagum y R. Menon, "OpenMP: an industry standard API for shared-memory programming," *IEEE Comput. Sci. Eng.*, vol. 5, núm. 1, pp. 46–55, ene.–mar. 1998.
- [3] R. Chandra, L. Dagum, D. Kohr, R. Menon, D. Maydan, y J. McDonald, *Parallel Programming in OpenMP*. San Francisco, CA: Morgan Kaufmann, 2001.
- [4] A. K. Jain, "Data clustering: 50 years beyond K-means," *Pattern Recognit. Lett.*, vol. 31, núm. 8, pp. 651–666, jun. 2010.
- [5] R. A. Fisher, "The use of multiple measurements in taxonomic problems," *Ann. Eugenics*, vol. 7, núm. 2, pp. 179–188, 1936.
- [6] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," en *Proc. AFIPS Spring Joint Comput. Conf.*, vol. 30, abr. 1967, pp. 483–485.